

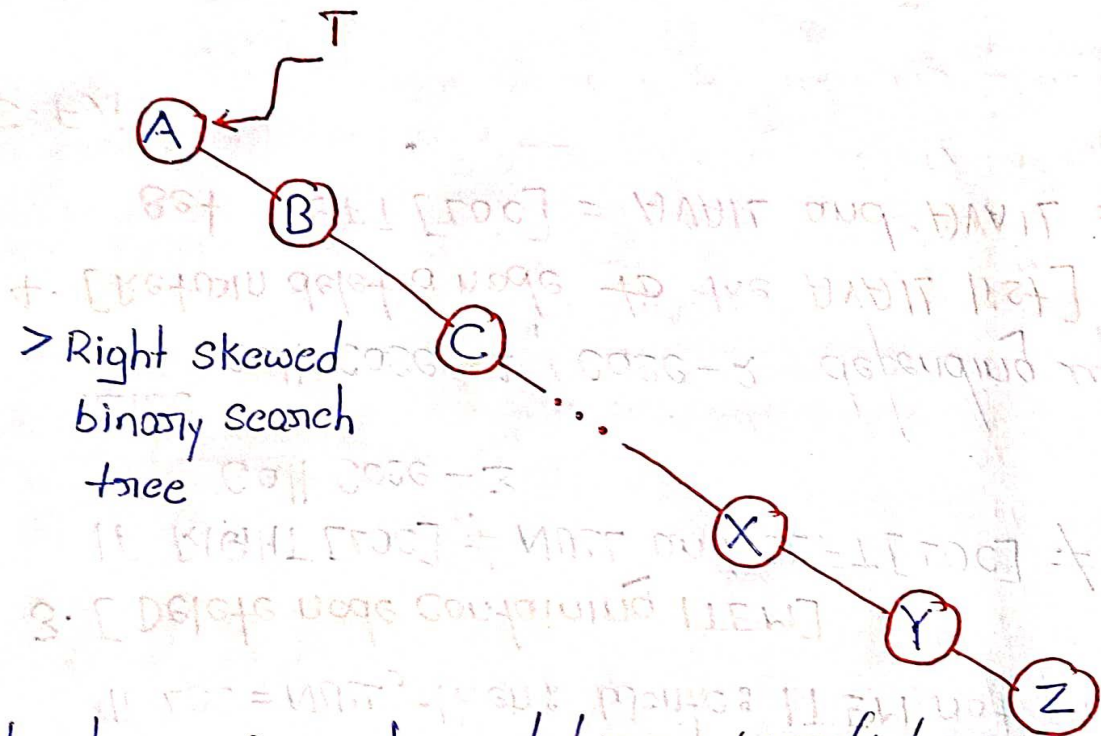
Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

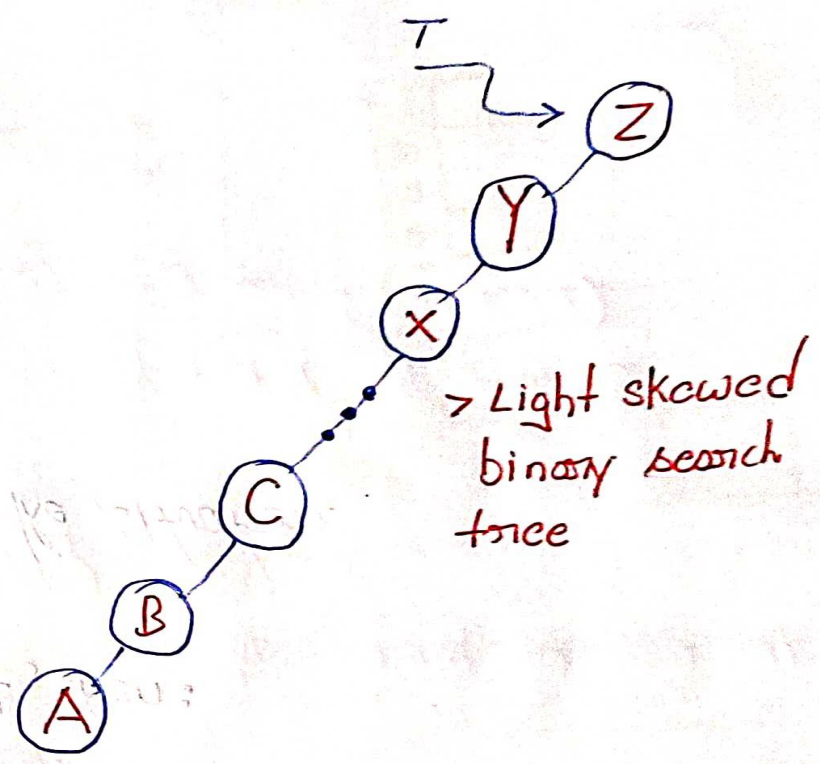
- Dr Rahul Mishra
- IIT Patna

- **Lecture 15**

AVL SEARCH TREES :



> Right skewed binary search tree



> Light skewed binary search tree

- > The disadvantage of a skewed binary search tree is that the worst case time complexity of a search is $O(n)$.
- > These arises the need to maintain the binary search tree to be of balanced height.
- > By doing so it is possible to obtain for the search operation a time complexity of $O(\log n)$ in the worst case.
- > One of the more popular balanced trees was by Adelson - Velskii and Landis (AVL)

Definition :-

- An empty binary tree is an AVL tree.
- Given T^L and T^R to be the left and right subtrees of T and $h(T^L)$ and $h(T^R)$ to be the heights of subtrees T^L and T^R respectively.

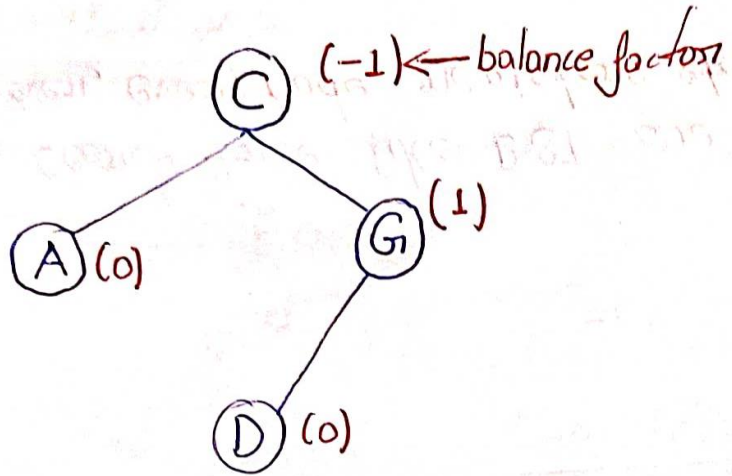
- T^L and T^R are AVL trees and $|h(T^L) - h(T^R)| \leq 1$

Balance factor (BF)

- For an AVL tree the balance factor of a node can be either 0, 1, or -1.

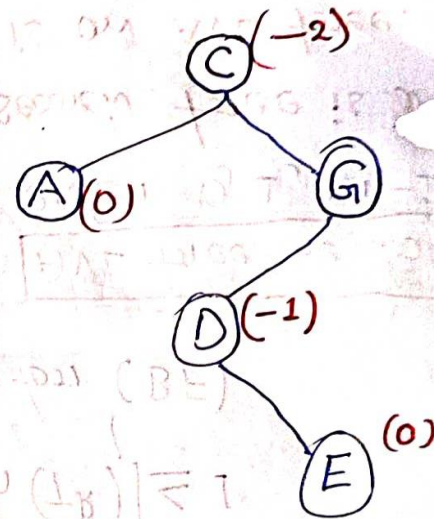
* AVL search tree is a binary search tree which is an AVL tree.

- AVL search tree like BST are represented using a linked representation. However, every node registers its balance factor.



> IF after insertion of the element, the balance factor of any node in the tree is affected
 → render the BST unbalanced

↳ IF we insert **E** in the previous tree.



Searching an AVL Search Tree →

* Searching an AVL search tree for an element is exactly similar to the method used in BST.

* Insertion in an AVL Search tree :-

> Inserting an element into an AVL search tree in its first phase is similar to that of the one used in BST.

> To overcome this situation an important technique is introduced namely Rotations.

→ To perform rotations it is necessary to identify a specific node A whose $BF(A)$ is neither 0, 1, or -1.

→ It is nearest ancestor to the inserted node on the path from inserted node to the root.

→ The rebalancing rotations are classified as LL, LR, RR, and RL.

LL rotation: Inserted node is the left subtree of left subtree of node A.

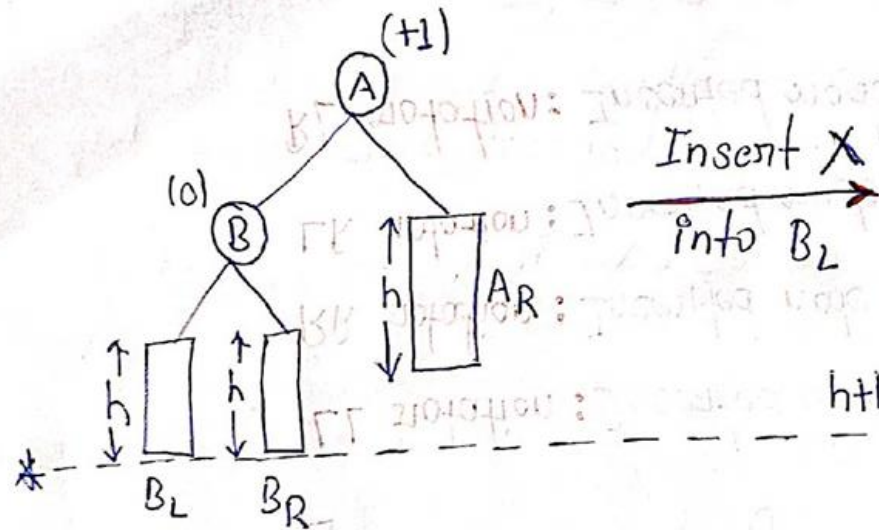
RR rotation: Inserted node is the right subtree of right subtree of node A.

LR rotation: Inserted node is the right subtree of left subtree of node A.

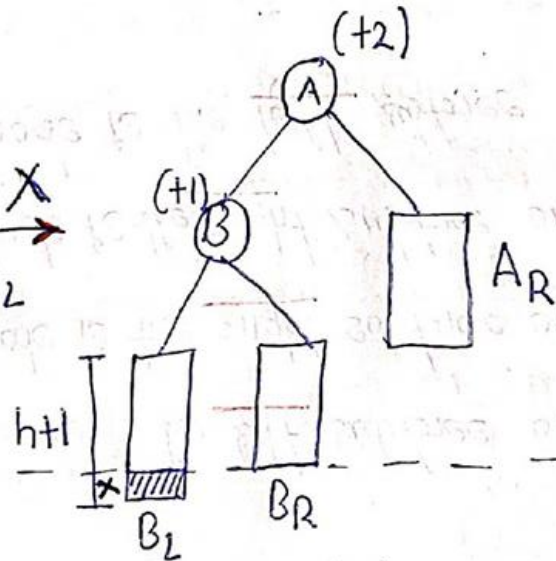
RL rotation: Inserted node is the left subtree of right subtree of node A.

LL - rotation :-

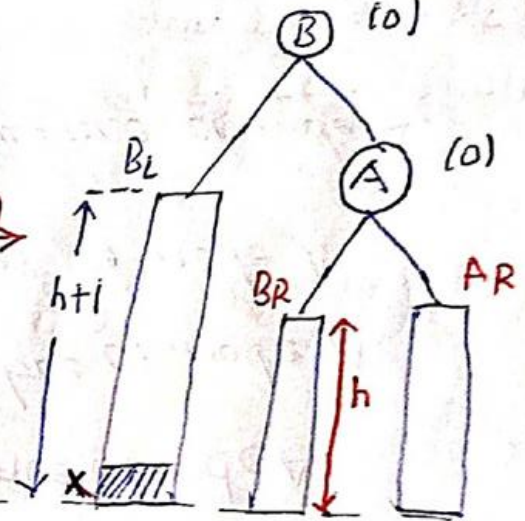
Balanced AVL search tree



Unbalanced AVL search tree after insertion



Balanced AVL search tree after rotation



B_L : Left subtree of B

B_R : Right subtree of B

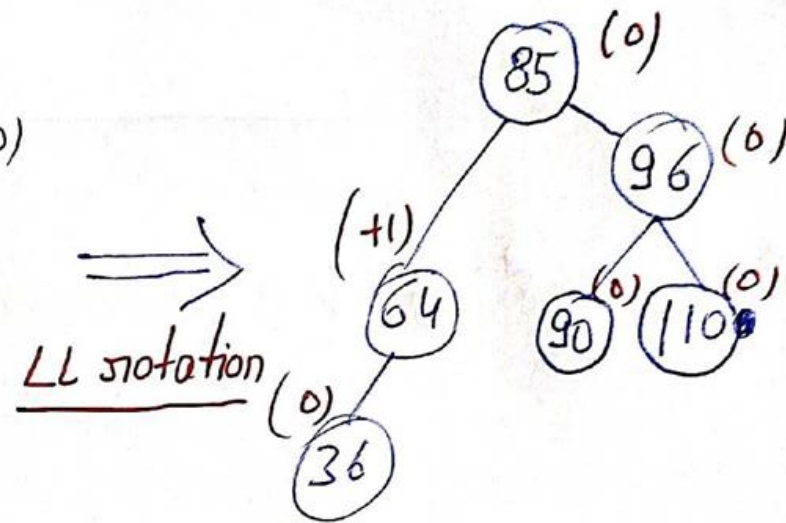
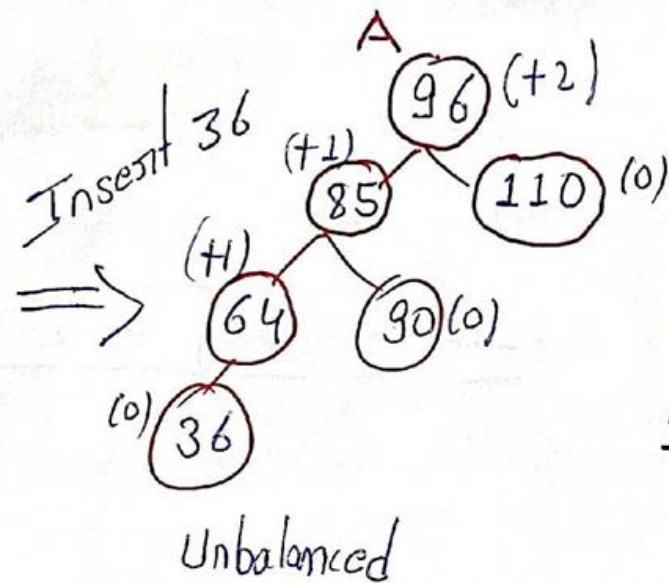
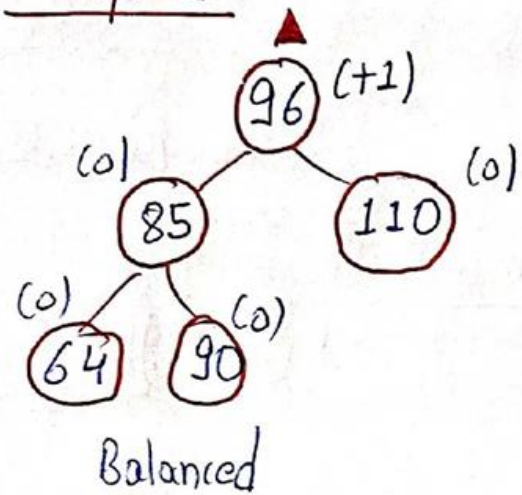
A_R : Right subtree of A

h : Height

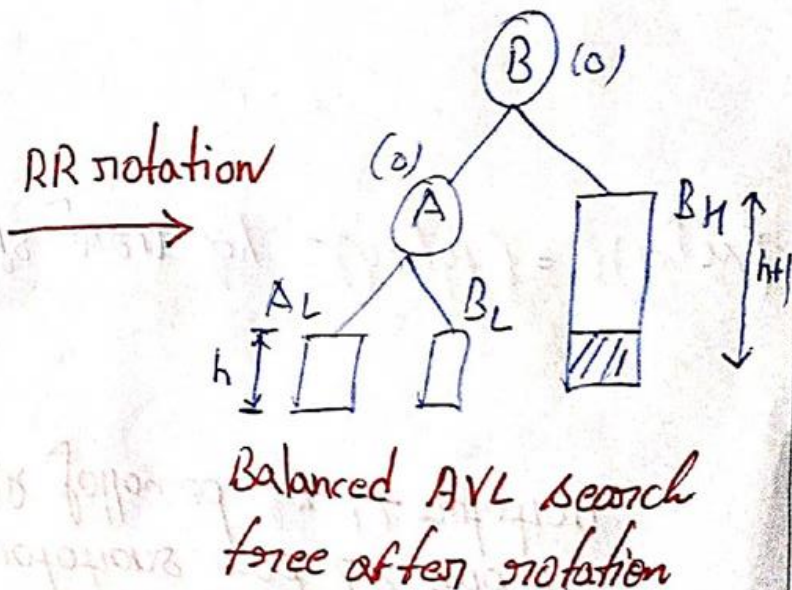
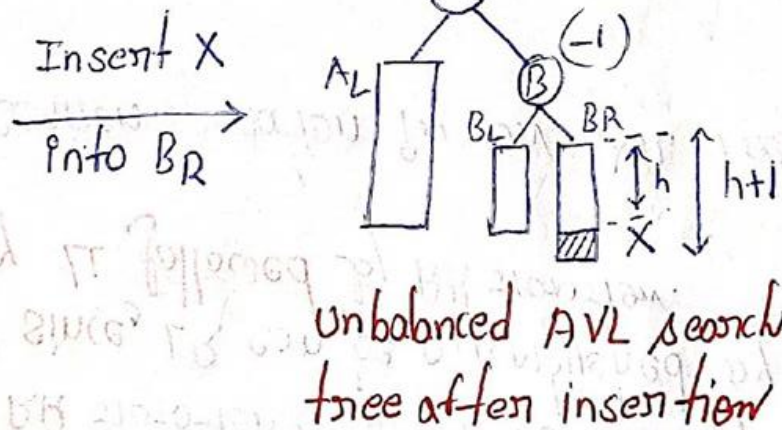
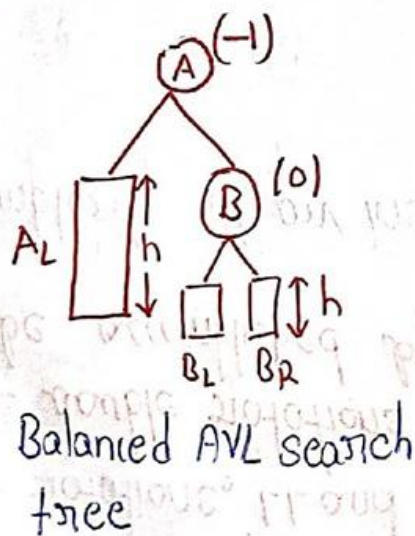
→ A new element (X) is inserted in the left subtree of left subtree of (A), the closest ancestor node whose $BF(A)$ becomes $+2$ after insertion.

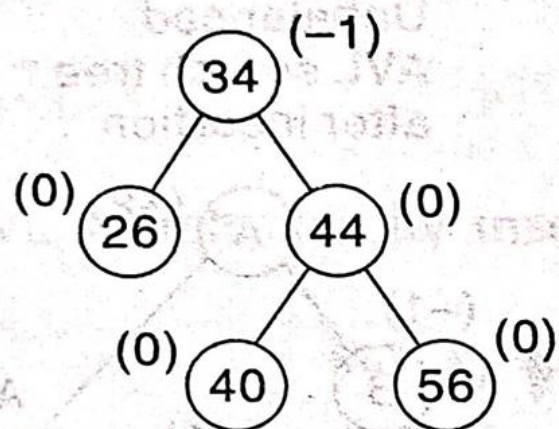
→ To rebalance the search tree, it is rotated so as to allow (B) to be root with B_L and A to be its left subtree and right child, respectively.

Example:1

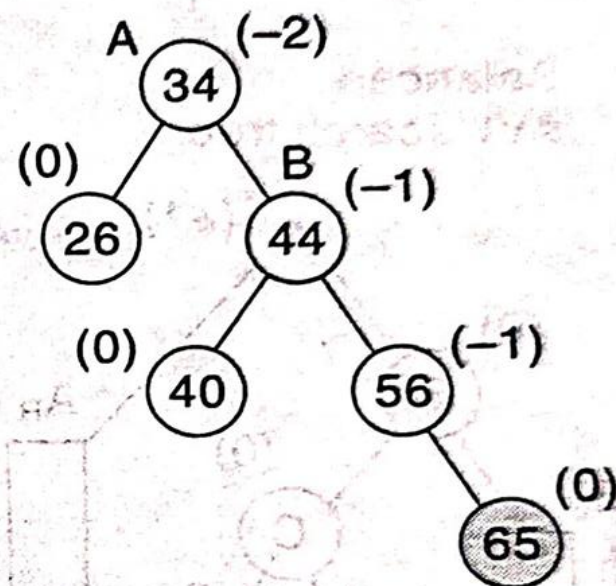
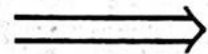


RR-notation:



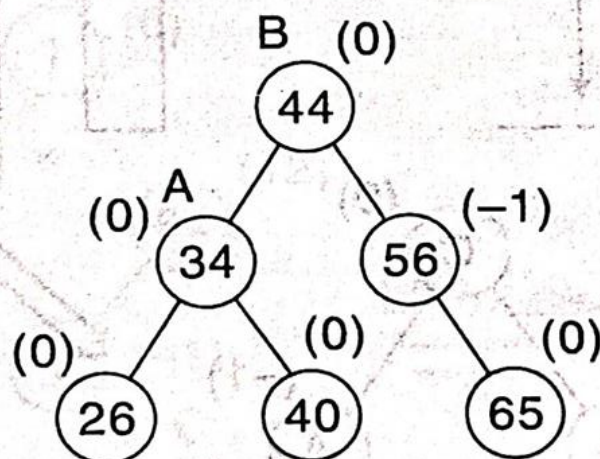


Insert 65



Unbalanced
AVL search tree
(b)

RR rotation



Balanced AVL search
tree after RR rotation
(c)

LR and RL rotations:-

- > The balancing methodology of LR and RL rotations are similar in nature but are mirror image of another.
- > Amongst the rotations, LL and RR rotations are called as single rotations and LR and RL are called as double rotations since, LR can be accomplished by RR followed by LL rotation and RL can be accomplished by LL followed by RR rotation.
- > The time complexity of any insertion operation in an AVL tree is given by $O(\text{height}) = O(\log n)$.

Deletion in an AVL Search Tree :-

In the event of an imbalance due to deletion, one or more rotations need to be applied to balance the AVL tree.

1. > RR Rotation:

IF $BF(B) = 0$, the RR rotation is executed.

2. > RL Rotation:

IF $BF(B) = 1$, the RL rotation is executed.

3. > RR-1 Rotation:

IF $BF(B) = -1$, the RR-1 rotation is executed.

m-Way Search Trees :-

- > All the data structures discussed so far favor data stored in the internal memory and hence support internal information retrieval.
- > To favor retrieval and manipulation of data stored in external memory, viz. **storage devices such as Hard disk**. It is required to have some specialized data structure.
- > 'm-way search tree', B-tree, and B⁺ tree are examples of such data structures.
- > m-way search trees are generalized versions of binary search trees. The goal of m-way search tree is to minimize the access while retrieving a key from a file.
- > An m-way search tree of height h calls for $O(h)$ number of access for an **insert/delete/retrieval** operation.
- > It pays to ensure that the height h is close to $\log_m(n+1)$.
- > It is because the number of elements in an m-way search tree of height h ranges from a minimum of h to maximum of $m^h - 1$.

Definition:

An m -way search tree T may be an empty tree. If T is not empty, it satisfies the following properties:

- (i) For some integer m , known as order of the tree, each node is of degree which can reach a maximum of m , in other words, each node has, at most m child nodes.
A node may be represented as $A_0, (K_1, A_1), (K_2, A_2), \dots, (K_{m-1}, A_{m-1})$ where $K_i, 1 \leq i \leq m-1$ are the keys and $A_i, 0 \leq i \leq m-1$ are the pointers to subtrees of T .
- (ii) If a node has K child nodes where $K \leq m$, then the node can have only $(K-1)$ keys $K_1, K_2, \dots, K_{K-1}$, contained in the node such that $K_i < K_{i+1}$ and each of the keys partitions all the keys in the subtrees in K subsets.
- (iii) For a node $A_0, (K_1, A_1), (K_2, A_2), \dots, (K_{m-1}, A_{m-1})$, all key values in the subtree pointed to by A_i are less than the key $K_{i+1}, 0 \leq i \leq m-2$ and all key values in the subtree pointed to by A_{m-1} are greater than K_{m-1} .
- (iv) Each of the subtrees $A_i, 0 \leq i \leq m-1$ are also m -way search trees.